Mawlana Bhashani Science and Technology University
Department of Information and Communication Technology

CampusShare

A Campus Resource Borrowing and Sharing Platform

Individual Capstone Project Proposal

Student Name: Swapnil Nag Swachha
Student ID: IT24001
Supervisor: Dr. Ziaur Rahman
Submission Date: 27 October 2026

Status: project proposal, system design and an initial UI prototype. The full application is planned for future development.

Contents

Abstract **3**

1 Introduction and Background **3**

2 Problem Statement **3**

3 Aim and Objectives **3**

4 Project Scope **4**

5 Stakeholders and User Roles **4**

6 Functional Requirements **4**

7 Non-Functional Requirements **6**

8 Feasibility Study **6**

9 Software Development Methodology **6**

10 Proposed System Architecture and Technology Stack **7**

11 System Design **7**

11.1 Data Flow Diagram (DFD) 7

11.2 Use Case Diagram 9

11.3 Activity Diagram 9

11.4 Entity Relationship Diagram (ERD) 10

11.5 Class Diagram 11

11.6 Object Diagram 12

12 Basic Database Design **13**

13 UI Prototype Overview **14**

14 Six-Week Development Schedule **15**

15 Milestones and Deliverables **16**

16 Basic Testing Strategy **16**

17 Risk Analysis and Mitigation **17**

18 Expected Outcomes **17**

19 Conclusion **17**

References **18**

Abstract

University students often need items for a short time, such as a scientific calculator for an exam, a lab coat for a practical session, or a textbook for one chapter. Buying these items for brief use is costly, while many other students own the same items and use them rarely. This proposal presents CampusShare, a simple web application that lets students list items they are willing to lend, browse and search available items, send borrowing requests, and track request and borrowing status. An administrator manages users, item categories, and reported items. The system is planned as a layered Java Spring Boot application with Spring MVC, Spring Data JPA, MySQL, and Thymeleaf views. The project follows the Incremental Model and is scheduled over six weeks for one student. This document contains the requirements, design diagrams, database design, schedule, testing strategy, and an initial UI prototype. The full backend has not yet been implemented.

1 Introduction and Background

Students on a campus share many small needs: textbooks, calculators, stationery, lab accessories, and sports equipment. Many of these items are used only during certain weeks of a semester. Informal sharing through messaging groups or notice boards exists, but requests are easily lost, availability is unclear, and nobody can see who has borrowed what.

CampusShare proposes a small, dedicated platform for this purpose. It is not intended to be a commercial marketplace. Its aim is to give students one organised place to offer items, find items, and manage borrowing in a clear and traceable way.

2 Problem Statement

Sharing items between students is currently informal and unstructured. As a result:

- students cannot easily find out which items are available on campus;
- borrowing requests are made through scattered messages and are easy to miss;
- owners cannot easily track who has their item and when it should be returned;
- there is no simple way to report a problem with a listing or to record feedback after borrowing.

A simple web-based system with clear request statuses is needed to organise this process.

3 Aim and Objectives

Aim: To design, and in later weeks implement, a simple web application that helps university students share and borrow campus resources in an organised way.

Objectives:

1. Analyse and document the requirements of the Student and Administrator roles.
2. Produce the system design: DFD, use case, activity, ERD, class, and object diagrams.
3. Design a relational database for users, items, categories, requests, transactions, reviews, and reports.
4. Implement item listing, browsing, and searching using Spring Boot and MySQL.

5. Implement the borrowing request, approval, borrowing, and return workflow.
6. Provide an administrator interface for users, categories, and reports.
7. Test the main workflows and deliver the documented project on GitHub.

4 Project Scope

Included	Excluded
User registration and login with two roles	Payment or deposit handling
Item listing, editing, and status management	Real-time chat or messaging
Browse, search, and filter by category and status	Artificial intelligence or recommendation features
Borrow request, approval or rejection, and return confirmation	Mobile native applications
Borrowing history and request tracking	Delivery or logistics tracking
Reviews after completed borrowing	Complex analytics and dashboards
Item reporting and administrator review	Cloud-scale deployment and microservices
Administrator management of users and categories	Integration with university systems

5 Stakeholders and User Roles

Stakeholder / Role	Description and main actions
Student (borrower)	Browses and searches items, submits and cancels borrow requests, views borrowing history, writes reviews, and reports items.
Student (owner)	Adds, edits, and removes own listings, approves or rejects requests, and confirms returns. Every student can act as both owner and borrower.
Administrator	Manages users (suspend or activate), manages item categories, and reviews reported items.
Project supervisor	Reviews the proposal, design, and final delivery.

6 Functional Requirements

ID	Requirement	Actor
FR-01	The system shall allow a user to register with a name, email, and password.	Student
FR-02	The system shall allow users to log in and log out, and shall restrict pages by role.	Student, Admin
FR-03	The system shall allow a student to add an item listing with title, description, category, condition, and pickup location.	Student
FR-04	The system shall allow a student to edit a listing, mark it UNAVAILABLE, or remove it.	Student

ID	Requirement	Actor
FR-05	The system shall display a list of items that can be browsed.	Student
FR-06	The system shall allow searching by keyword and filtering by category and status.	Student
FR-07	The system shall show item details, including owner, category, condition, and status.	Student
FR-08	The system shall allow a student to submit a BorrowRequest with proposed dates and a message for an AVAILABLE item.	Student
FR-09	The system shall allow a borrower to cancel their own PENDING request.	Student
FR-10	The system shall allow an owner to approve or reject a PENDING request for their item.	Student
FR-11	When a request is approved, the system shall create an ACTIVE BorrowTransaction and set the item to BORROWED.	System
FR-12	The system shall allow the owner to confirm return, setting the transaction to RETURNED and the item to AVAILABLE.	Student
FR-13	The system shall mark an ACTIVE transaction as OVERDUE when its due date has passed without return.	System
FR-14	The system shall show a student their own listings, requests, and borrowing history with statuses.	Student
FR-15	The system shall allow a borrower to submit a Review (rating 1–5 and comment) for a returned transaction.	Student
FR-16	The system shall allow a student to report an item with a reason and details.	Student
FR-17	The system shall allow an administrator to view users and suspend or activate accounts.	Admin
FR-18	The system shall allow an administrator to add, rename, and remove item categories.	Admin
FR-19	The system shall allow an administrator to review reports and mark them RESOLVED or DISMISSED.	Admin

7 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Usability	Main actions should be reachable within a few clicks, with clear status labels.
NFR-02	Security	Passwords shall be stored as hashes; pages shall be protected by role; inputs shall be validated.
NFR-03	Performance	Normal pages should load within a few seconds on a typical campus network for the expected small user base.
NFR-04	Reliability	Data shall be stored in MySQL with foreign key constraints to keep records consistent.
NFR-05	Maintainability	Code shall follow a layered structure (controller, service, repository, entity).
NFR-06	Compatibility	The interface shall be responsive and work in current desktop and mobile browsers.

8 Feasibility Study

Type	Assessment
Technical	Java, Spring Boot, MySQL, and Thymeleaf are widely documented, free, and suitable for a single developer. The planned features use standard CRUD operations and one status-based workflow.
Operational	The workflow (list, request, approve, return) follows how students already share items informally, so little training is expected. An administrator role keeps content manageable.
Economic	All planned tools are free or open source. No paid services are required, and the system can run on a personal computer during development.
Schedule	The scope is limited to the features above and is divided into small increments. Six weeks is considered achievable for one student, with Week 6 kept mainly for testing and documentation.

9 Software Development Methodology

The project will follow the **Incremental Model**. The system is built as a sequence of working increments, each adding a part of the functionality [5, 4].

Justification:

- The requirements are fairly clear, and the features divide naturally into increments.
- A working core (item listing) is available early and can be tested before more features are added.
- Risk is reduced, because problems are found in small parts rather than at the end.
- It suits one developer working under a fixed deadline.

Increment	Weeks	Content
Increment 1	2–3	Project setup, database, accounts, and core item listing, browsing, and search.
Increment 2	4	Borrow request, approval or rejection, borrowing, and return workflow.
Increment 3	4–5	Reviews, reports, administrator functions, and UI integration.

10 Proposed System Architecture and Technology Stack

CampusShare will be a server-side rendered web application with a simple layered architecture:

- **Presentation layer:** Thymeleaf templates with HTML, CSS, and Bootstrap.
- **Controller layer:** Spring MVC controllers handle requests and return views.
- **Service layer:** business rules, such as approving a request and updating item status.
- **Repository layer:** Spring Data JPA repositories for database access.
- **Entity layer:** JPA entities mapped by Hibernate to MySQL tables.

Area	Technology
Language and framework	Java, Spring Boot, Spring MVC [8]
Data access	Spring Data JPA, Hibernate [9]
Database	MySQL [3]
Views and styling	Thymeleaf, HTML, CSS, Bootstrap, JavaScript [7, 6]
Build and version control	Maven, Git, GitHub

11 System Design

This section presents the six required diagrams. Their source files are stored in the `diagrams/` folder of the repository. All diagrams use the same class names and statuses: item status (AVAILABLE, BORROWED, UNAVAILABLE), request status (PENDING, APPROVED, REJECTED, CANCELLED), and transaction status (ACTIVE, RETURNED, OVERDUE).

11.1 Data Flow Diagram (DFD)

Figure 1 shows the context diagram, and Figure 2 shows the Level 1 DFD with processes, data stores, and external entities. Source files: `dfd_context.mmd`, `dfd_level1.mmd`.

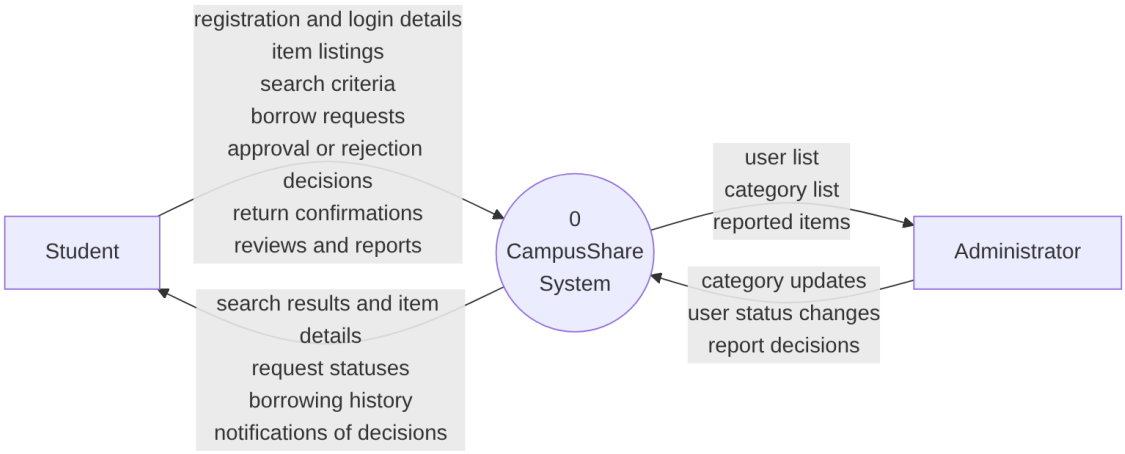


Figure 1: DFD context diagram (Level 0).

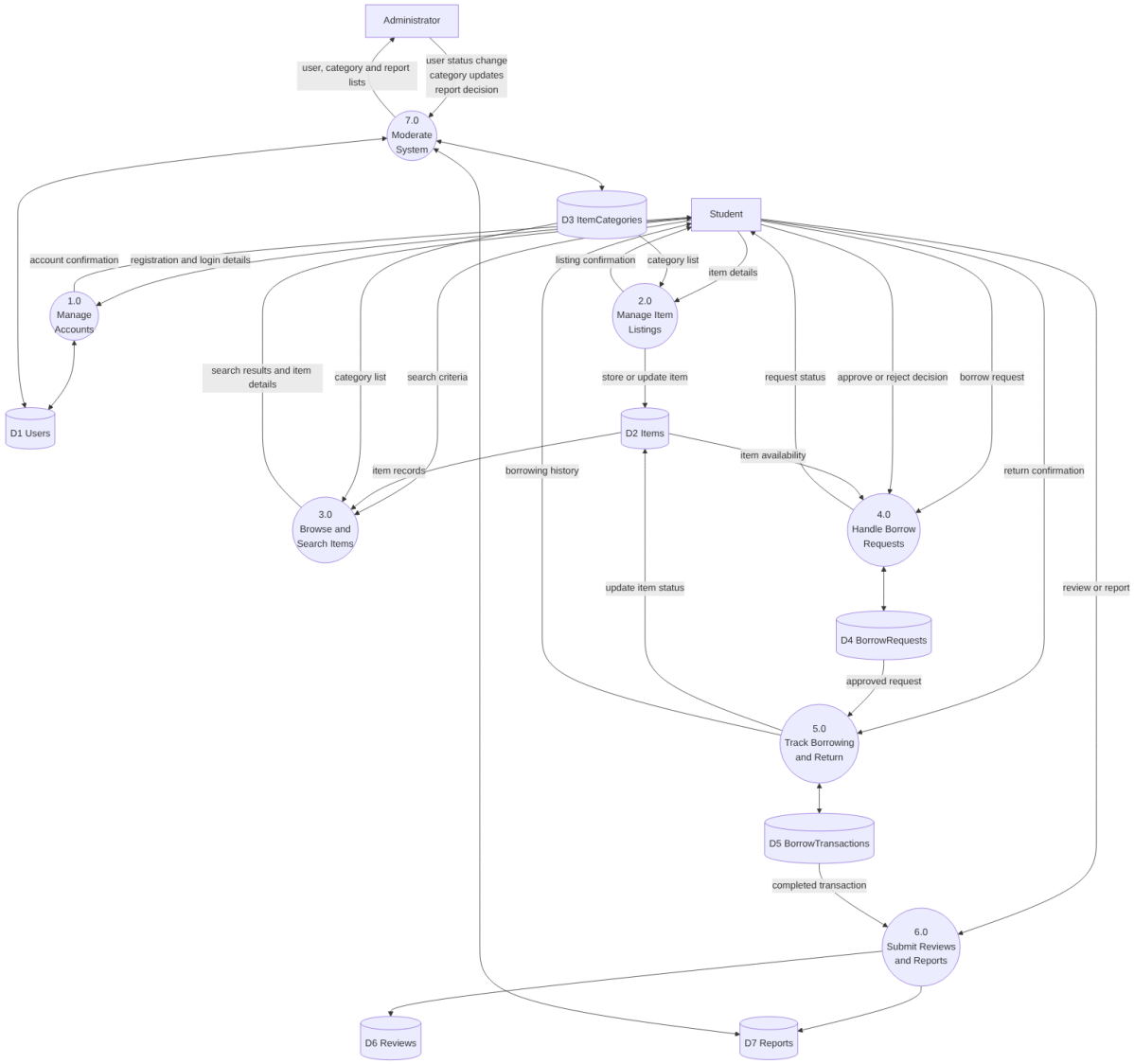


Figure 2: DFD Level 1: processes 1.0–7.0 and data stores D1–D7.

11.2 Use Case Diagram

Figure 3 shows the actions available to the Student and Administrator actors (source: `use_case.puml`) [1, 2].



Figure 3: UML use case diagram.

11.3 Activity Diagram

Figure 4 shows the borrowing process from browsing an item to return, including the approval or rejection decision (source: `activity.puml`).

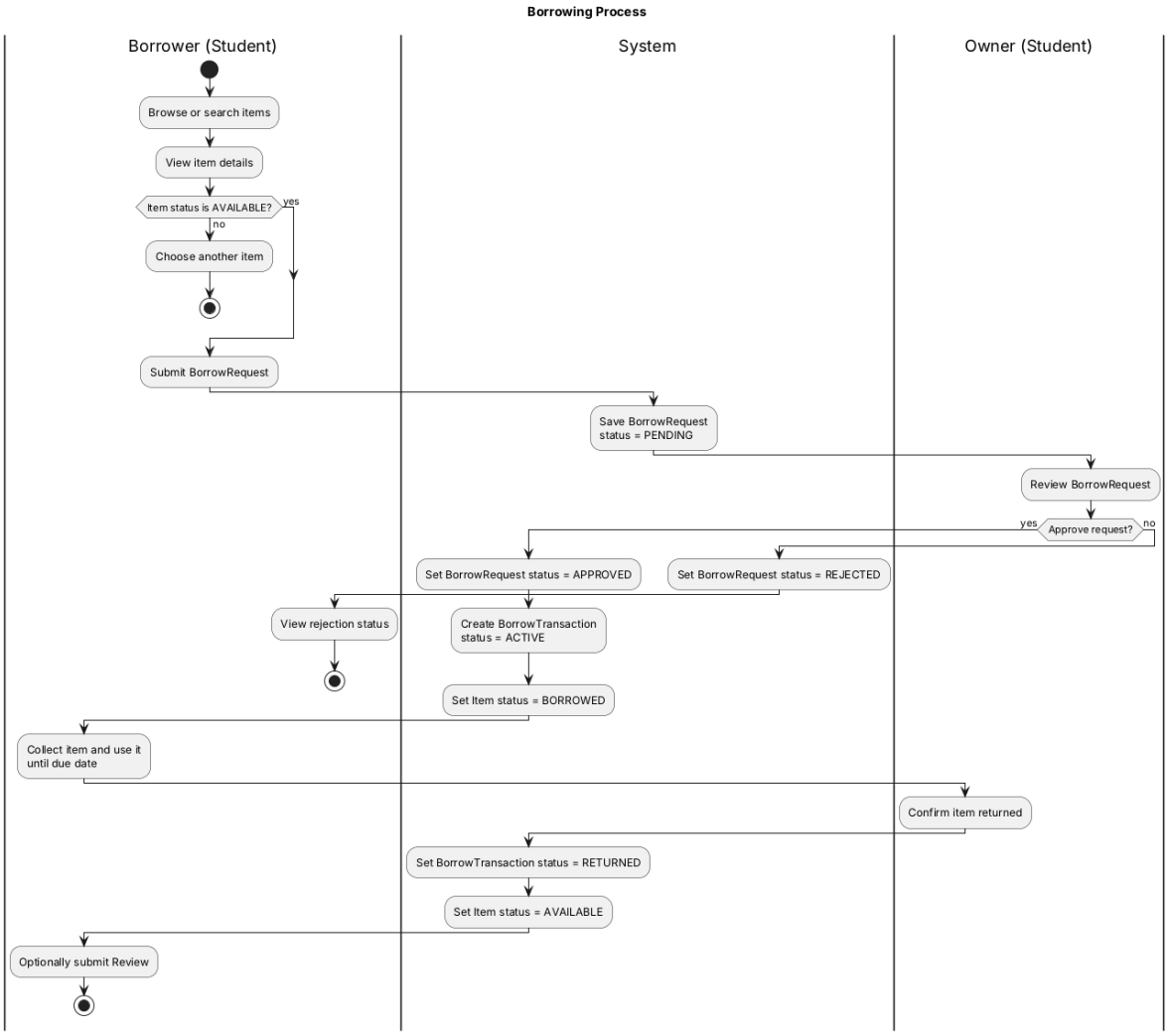


Figure 4: UML activity diagram of the borrowing process.

11.4 Entity Relationship Diagram (ERD)

Figure 5 shows the database entities with primary keys (PK), foreign keys (FK), and relationships (source: `erd.mmd`).

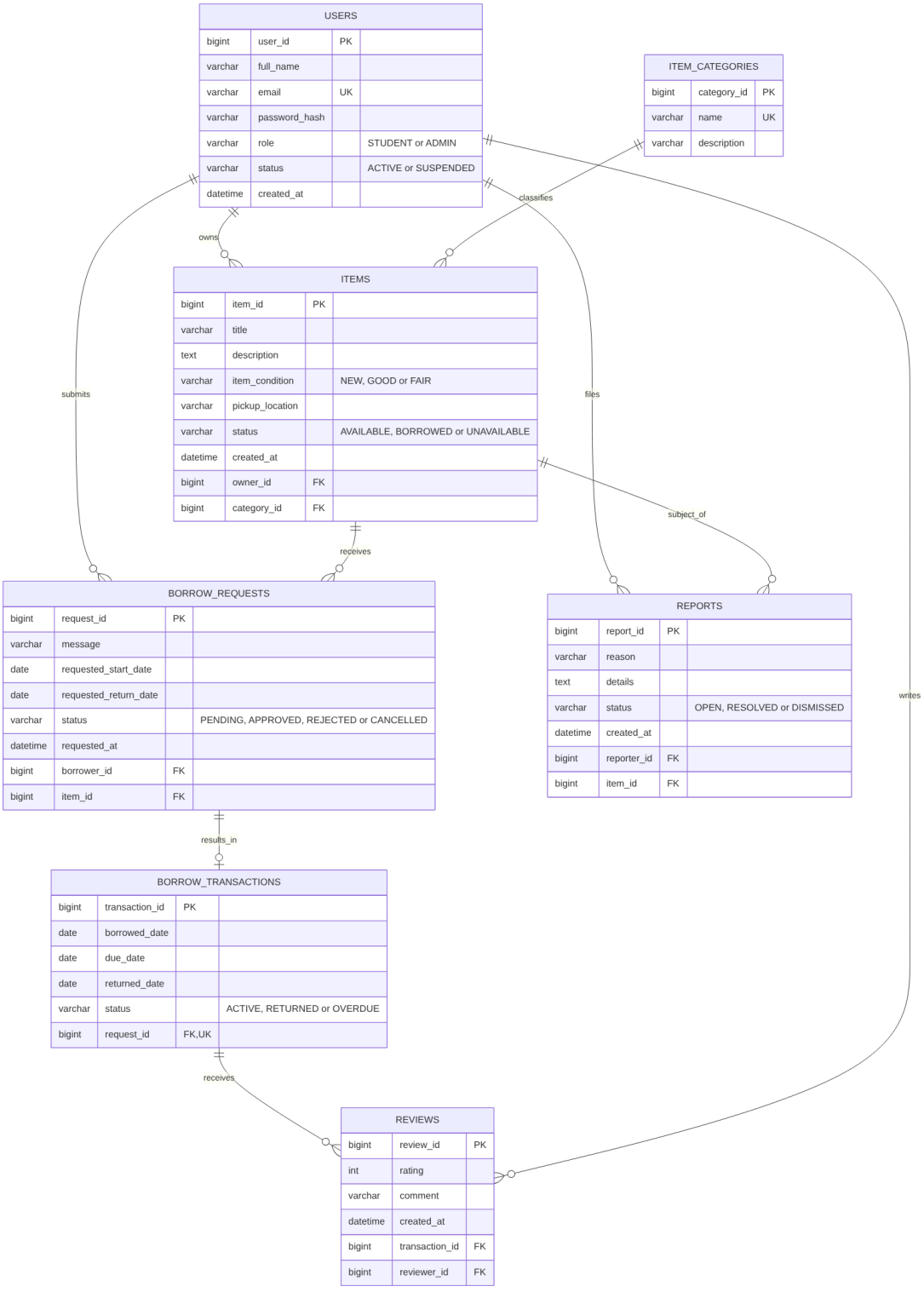


Figure 5: Entity relationship diagram.

11.5 Class Diagram

Figure 6 shows the seven domain classes (User, Item, ItemCategory, BorrowRequest, BorrowTransaction, Review, Report), their attributes, selected methods, multiplicities, and status enumerations (source: `class_diagram.puml`).

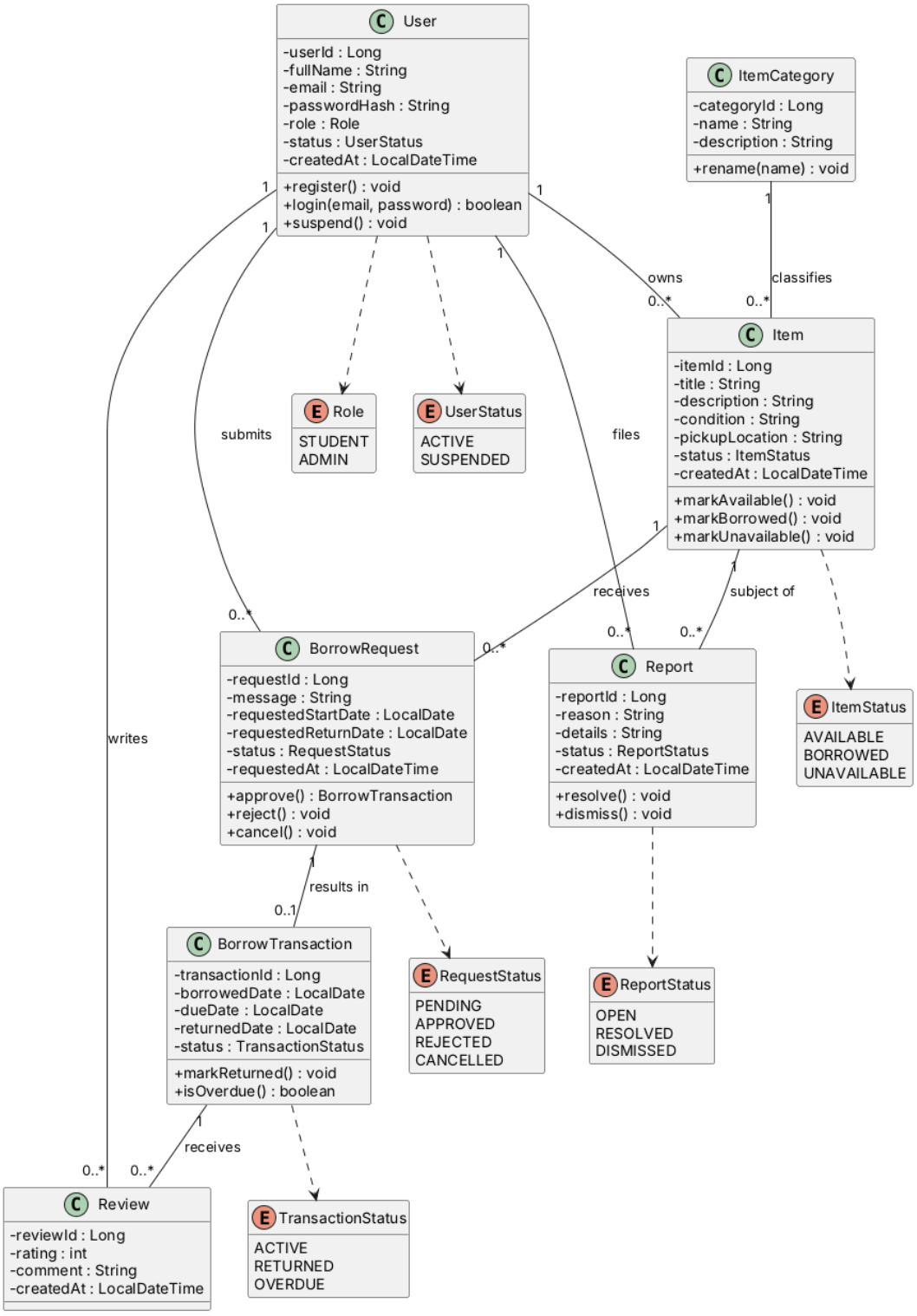


Figure 6: UML class diagram of the domain model.

11.6 Object Diagram

Figure 7 shows one example state of the system: Amina Rahman owns a calculator, Tanvir Hasan has an approved request for it, and an ACTIVE transaction exists (source: `object_diagram.puml`). The values are sample data only.

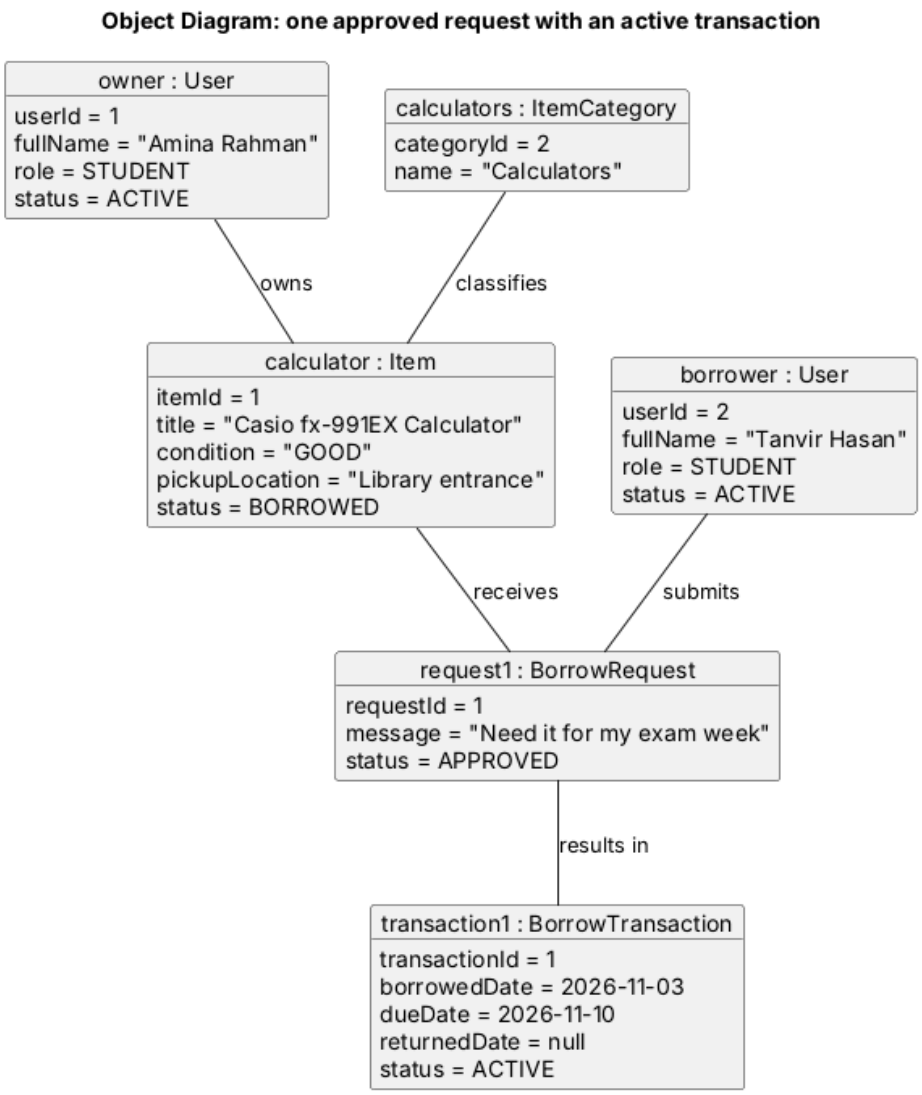


Figure 7: UML object diagram (an instance snapshot, not a class model).

12 Basic Database Design

The MySQL database will contain seven tables that correspond to the domain classes. Table 2 summarises them; the full structure is shown in the ERD.

Table 2: Planned database tables.

Table	Class	Key columns
users	User	user_id (PK), full_name, email (unique), password_hash, role, status
item_categories	ItemCategory	category_id (PK), name (unique), description
items	Item	item_id (PK), title, item_condition, status, owner_id (FK), category_id (FK)
borrow_requests	BorrowRequest	request_id (PK), status, borrower_id (FK), item_id (FK)
borrow_transactions	BorrowTransaction	transaction_id (PK), due_date, returned_date, status, request_id (FK, unique)
reviews	Review	review_id (PK), rating, comment, transaction_id (FK), reviewer_id (FK)
reports	Report	report_id (PK), reason, status, reporter_id (FK), item_id (FK)

13 UI Prototype Overview

An initial prototype is provided in the `prototype/` folder, built with HTML, CSS, Bootstrap, and JavaScript. It uses fixed sample data held in the browser and **is not connected to a backend or database**. Login, saving, and borrowing actions are simulated for demonstration only.

Table 3: Prototype screens and the domain classes they demonstrate.

Screen	Domain classes shown
Dashboard	User, Item, BorrowRequest, BorrowTransaction
Browse and search	Item, ItemCategory, User (owner)
Item details	Item, ItemCategory, User, Review, Report, BorrowRequest
Add or edit listing	Item, ItemCategory
Requests	BorrowRequest, Item, User, BorrowTransaction
My items and activity	Item, BorrowTransaction, BorrowRequest, Review

Implementation status

Item	Description	Status
Proposal and design diagrams	This document and the diagram source files	Prepared
UI prototype	Static, simulated screens with sample data	Prepared
Spring Boot application	Controllers, services, repositories, entities	Planned
MySQL database	Schema and persistence	Planned
Real login and security	Authentication and role-based access	Planned
Testing	Test cases listed in this proposal	Planned

14 Six-Week Development Schedule

The schedule below is a **proposed plan for future work**. It does not mean that any of the listed development tasks, other than the proposal, diagrams, and prototype, have been completed. Relative weeks are used instead of calendar dates.

Table 4: Task schedule.

ID	Task	Start	End	Duration	Depends on
T1	Requirements and proposal	W1	W1	1 week	–
T2	System design and UML diagrams	W1	W2	1.5 weeks	T1
T3	Database and project setup	W2	W2	1 week	T2
T4	Core item listing functionality	W3	W3	1 week	T3
T5	Borrowing request and approval workflow	W4	W4	1 week	T4
T6	UI integration	W4	W5	1.5 weeks	T4, T5
T7	Testing and debugging	W5	W6	1.5 weeks	T5, T6
T8	Documentation and final delivery	W6	W6	1 week	T7

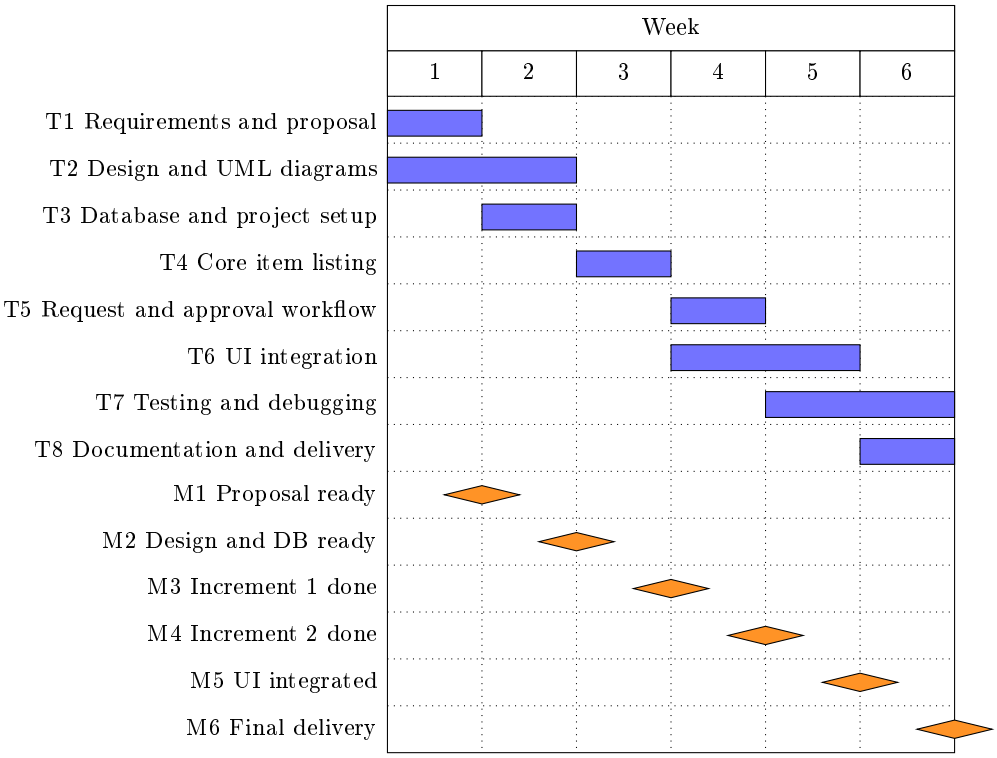


Figure 8: Proposed Gantt chart (relative weeks). Blue bars are tasks; orange diamonds are milestones.

15 Milestones and Deliverables

ID	End of	Milestone	Deliverable
M1	W1	Proposal and requirements ready	Proposal document, requirements list
M2	W2	Design and database ready	Six diagrams, MySQL schema, Maven project skeleton
M3	W3	Increment 1 complete	Working item listing, browsing, and search
M4	W4	Increment 2 complete	Working request, approval, and return workflow
M5	W5	UI integrated	Thymeleaf pages for all main screens, reviews, reports, and admin pages
M6	W6	Final delivery	Tested application, final report, GitHub repository, README

16 Basic Testing Strategy

Testing is planned for Weeks 5–6, with simple checks after each increment. The strategy includes:

- **Unit tests** for service methods using JUnit (for example, approving a request);
- **Manual functional tests** of each workflow through the browser;
- **Basic validation tests** for forms and role-based access.

Table 5 lists example test cases. They are planned and have not yet been run.

Table 5: Example planned test cases.

ID	Test	Expected result
TC-01	Register with an already used email	Registration is rejected with a message
TC-02	Add a listing with an empty title	Form shows a validation error
TC-03	Search items by keyword “calculator”	Only matching items are listed
TC-04	Request an item with status BORROWED	Request is not allowed
TC-05	Owner approves a PENDING request	Request becomes APPROVED, an ACTIVE BorrowTransaction is created, and the item becomes BORROWED
TC-06	Owner confirms return	Transaction becomes RETURNED and the item becomes AVAILABLE
TC-07	Student opens an admin page	Access is denied

17 Risk Analysis and Mitigation

ID	Risk	Level	Mitigation
R1	Limited time for one developer	High	Keep scope small; build the core workflow first; cut optional features if needed.
R2	Difficulty with Spring Boot or JPA	Medium	Follow official documentation; build small examples early in Week 2.
R3	Workflow errors in request statuses	Medium	Define status rules in the diagrams; write unit tests for the service layer.
R4	Data loss or code loss	Medium	Commit to GitHub regularly; keep a database script.
R5	Security mistakes in login or access	Medium	Hash passwords; use role checks; validate inputs.
R6	Last-week delays	Medium	Reserve Week 6 for testing and documentation only.

18 Expected Outcomes

At the end of the planned development, the project is expected to deliver:

- a working web application for listing, browsing, requesting, and returning campus items;
- an administrator interface for users, categories, and reports;
- a MySQL database design consistent with the ERD and class diagram;
- a tested core workflow and a documented GitHub repository;
- a final report describing the design, implementation, and testing.

These outcomes are targets of the plan and depend on completing the proposed schedule.

19 Conclusion

CampusShare addresses a small but practical problem: organising item sharing among students. The project is limited in scope, uses well-documented technologies, and is divided into increments suitable for one student over six weeks. This proposal provides the requirements, six design diagrams, database design, schedule, and an initial prototype as a base for the implementation phase.

References

[1] Martin Fowler. *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. Addison-Wesley, 3 edition, 2003.

[2] Object Management Group. Unified modeling language (uml), version 2.5.1. `https://www.omg.org/spec/UML/2.5.1`, 2017.

[3] Oracle Corporation. Mysql reference manual. `https://dev.mysql.com/doc/`.

[4] Roger S. Pressman and Bruce R. Maxim. *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education, 9 edition, 2019.

[5] Ian Sommerville. *Software Engineering*. Pearson, 10 edition, 2016.

[6] The Bootstrap Authors. Bootstrap documentation. `https://getbootstrap.com/docs/`.

[7] The Thymeleaf Team. Thymeleaf documentation. `https://www.thymeleaf.org/documentation.html`.

[8] VMware / Broadcom (Spring Team). Spring boot reference documentation. `https://docs.spring.io/spring-boot/index.html`.

[9] VMware / Broadcom (Spring Team). Spring data jpa reference documentation. `https://docs.spring.io/spring-data/jpa/reference/`.